arXiv:2312.07859v3 [cs.LG] 27 Jan 2025

Fine-grained Graph Rationalization

Zhe Xu[1], **Menghai Pan**[2], **Yuzhong Chen**[2], **Huiyuan Chen**[2], **Yuchen Yan**[1], **Mahashweta Das**[2], **Hanghang Tong**[1]

[1]University of Illinois Urbana-Champaign
[2]Visa Research
Corresponding: zhexu3@illinois.edu

Abstract

Rationale discovery is defined as finding a subset of the input data that maximally supports the prediction of downstream tasks. In the context of graph machine learning, graph rationale is defined as locating the critical subgraph in the given graph topology. In contrast to the rationale subgraph, the remaining subgraph is named the environment subgraph. Graph rationalization can enhance the model performance as the mapping between the graph rationale and prediction label is viewed as invariant, by assumption. To ensure the discriminative power of the extracted rationale subgraphs, a key technique named *intervention* is applied whose heart is that given changing environment subgraphs, the semantics from the rationale subgraph is invariant, which guarantees the correct prediction result. However, most, if not all, of the existing graph rationalization methods develop their intervention strategies on the graph level, which is coarse-grained. In this paper, we propose FIne-grained Graph rationalization (FIG). Our idea is driven by the self-attention mechanism which provides rich interactions between input nodes. Based on that, FIG can achieve node-level and virtual node-level intervention. Our experiments involve 7 real-world datasets, and the proposed FIG shows significant performance advantages compared to 13 baseline methods.

1 Introduction

Rationale refers to a subset of the input features that play a crucial role in model predictions for downstream tasks [Chang *et al.*, 2020; Liu *et al.*, 2022; Wu *et al.*, 2022a; Zhang *et al.*, 2022; Yue *et al.*, 2024; Sui *et al.*, 2023]. In the context of graph machine learning, graph rationale is defined as a subgraph of the input graph containing the most task-relevant semantics.

The application of graph rationale is broad, for example, it can greatly enhance model performance for graph-level tasks [Wu *et al.*, 2022a] by identifying the key components of the input graph. Additionally, the discovery of rationales can improve model explainability [Liu *et al.*, 2022], as it highlights the parts of the input graph that significantly contribute to the final prediction.

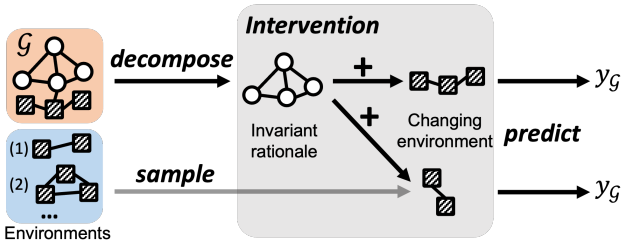


Figure 1: Illustration of the rationale/environment decomposition and intervention. Round nodes denote graph rationales, and square nodes (with stripes) denote the environments. The intervention aims to ensure the rationale from graph $\mathcal{G}$ truly has the discriminative power for the label $y_{\mathcal{G}}$.

Existing graph rationalization solutions [Liu *et al.*, 2022; Wu *et al.*, 2022a] employ a trainable *augmenter* to execute the rationale/environment decomposition. In this process, a node/edge mask is generated by the augmenter to decompose the given graph into a rationale graph and an environment graph. Inspired by the content-style decomposition [Kaddour *et al.*, 2022], the key idea of graph rationalization is to preserve the utility of the graph rationale even when faced with changing environment graphs (see Figure 1). To achieve this, a technique named *intervention* is used, where the environment graph interacts with the rationale graph.

The intervention mechanism (named *intervener*) is essential in the graph rationalization process, as it must accurately represent the interaction between the rationale and the environment. Intuitively, the intervener should work in an adversarial behavior against the aforementioned augmenter, a point not emphasized in the existing literature. If the intervener is more powerful, it can capture more detailed interactions between the rationale and environment subgraphs. Given such a powerful intervener, the augmenter is compelled to minimize these interactions between the graph rationale and the environment to obtain a "purer" graph rationale.

Unfortunately, existing works develop interveners in a coarse and non-parametric manner. After performing rationale/environment decomposition on the graph data, they com-

pute graph-level embeddings for the rationale and environment subgraphs. The intervention is then designed as an interaction between these graph-level embeddings. For example, [Liu *et al.*, 2022] adds the environment embedding into the rationale embedding as the intervened rationale embedding; [Wu *et al.*, 2022a] defines the intervened prediction as the Hadamard product between the predictions based on the rationale subgraph and the environment subgraph. We argue that such a graph-level non-parametric intervention is insufficient to represent the interaction between the rationale and environment graphs effectively.

In response to this limitation, we propose a fine-grained, parametric intervention mechanism named FIne-grained Graph rationalization (FIG). Our proposed FIG draws inspiration from the self-attention module in the Transformer model, which captures interactions between input tokens. Building upon insights from Transformer [Vaswani *et al.*, 2017] and its linear variant Linformer [Wang *et al.*, 2020], FIG formulates the interaction between the rationale and environment subgraphs at the node-level or the virtual node-level. The two variants are named FIG-N and FIG-VN. Additionally, to maximize the effectiveness of the intervention, we formulate a min-max game involving the node encoder, augmenter, intervener, and predictor, compelling the rationale subgraph to be as informative as possible.

We conduct comprehensive experiments on 7 graph-level benchmarks to evaluate the proposed approach and compare FIG-N/VN against 13 state-of-the-art baseline methods. The results demonstrate that our proposed FIG and its variants outperform the baseline methods, validating their superior performance. Our primary contributions in this paper are summarized as follows:

- We address the graph rationalization problem via a fine-grained model FIG, which works at the node/virtual node level.
- A min-max objective function is proposed so that the effectiveness of the proposed intervener can be maximized.
- Extensive experiments covering 13 baseline methods and 7 real-world datasets are conducted.

2 Preliminaries

This section first introduces the notations used throughout this paper 2.1. Then, the classic Transformer architecture is introduced in Section 2.2, whose self-attention module is the important building block of our model. Last but not least, we introduce the overall ideas of existing graph rationalization works in Section 2.3.

2.1 Notations

We adopt the following notation conventions: bold uppercase letters for matrices and tensors (e.g., $\mathbf{A}$), bold lowercase letters for column vectors (e.g., $\mathbf{u}$), lowercase and uppercase letters in regular font for scalars (e.g., d, K), and calligraphic letters for sets (e.g., $\mathcal{T}$). To index vectors/matrices/tensors, we follow the syntax from NumPy[1] (0-based). Specifically, $\mathbf{A}[p,:]$ and $\mathbf{A}[:,q]$ represent the p-th row and the q-th column of matrix $\mathbf{A}$ respectively; $\mathbf{A}[p,q]$ represents the entry at the p-th row and the q-th column. Similarly, $\mathbf{u}[p]$ denotes the p-th entry of vector $\mathbf{u}$. In addition, the slicing syntax for vectors/matrices/tensors is used. For example, for a matrix $\mathbf{A}$, $\mathbf{A}[i:j,:]$ denotes rows from the i-th row (included) to the j-th row (excluded) and $\mathbf{A}[:,:k]$ denotes all the columns before the k-th column. The superscript $\top$ denotes the transpose of matrices and vectors. $\odot$ represents the Hadamard product, and $\circ$ denotes function composition. We use $||$ to represent the concatenation operation and the specific dimension of concatenation will be clarified based on the context.

An attributed graph can be represented as $\mathcal{G} = (\mathbf{A}, \mathbf{X}, \mathbf{E})$, where $\mathbf{A} \in \mathbb{R}^{n \times n}$ is the adjacency matrix, $\mathbf{X} \in \mathbb{R}^{n \times d_X}$ is the node feature matrix, and $\mathbf{E} \in \mathbb{R}^{n \times n \times d_E}$ is the edge feature tensor. Here, n denotes the number of nodes, and d_X (or d_E) represents the dimensions of node (or edge) features, respectively. This paper assumes the node and edge feature dimensions are the same (i.e., $d_X = d_E = d$) for brevity; if they differ, a simple, fully connected layer can map them into a common feature space. Our main focus in this paper is on graph property prediction tasks. The ground truth of a graph is represented by y.

2.2 Graph Transformer

The core modules of the Transformer architecture [Vaswani *et al.*, 2017] are the self-attention layer and the feed-forward network layer. Given the input as a sequence of symbol representations $\mathbf{H} \in \mathbb{R}^{n \times d_H}$, it is first transformed into the query, key, and value matrices as

$$\mathbf{Q} = \mathbf{H}\mathbf{W}_Q, \mathbf{K} = \mathbf{H}\mathbf{W}_K, \mathbf{V} = \mathbf{H}\mathbf{W}_V, \tag{1}$$

where $\mathbf{W}_Q \in \mathbb{R}^{d_H \times d_Q}$, $\mathbf{W}_K \in \mathbb{R}^{d_H \times d_K}$, $\mathbf{W}_V \in \mathbb{R}^{d_H \times d_V}$. For the brevity of the presentation, we set $d_H = d_Q = d_K = d_V = d$. Then, the self-attention module works as,

$$\mathbf{P} = \texttt{Attn}(\mathbf{H}) = \sigma(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d}}), \tag{2a}$$

$$\mathbf{H} \leftarrow \mathbf{P}\mathbf{V} + \mathbf{H}. \tag{2b}$$

Typically, the non-linearity σ is `Softmax`. The feed-forward network (FFN) updates the symbol representations $\mathbf{H}$ as:

$$\mathbf{H} \leftarrow \texttt{FFN}(\mathbf{H}) + \mathbf{H}. \tag{3}$$

Additional techniques such as layer/batch normalization [Ba *et al.*, 2016; Ioffe and Szegedy, 2015], dropout [Srivastava *et al.*, 2014], and multi-head attention [Vaswani *et al.*, 2017] can be included, but omitted here for brevity.

While Transformers was originally devised for sequence or set data with positional encoding, numerous techniques have since been introduced to adapt Transformers for graph data. Based on the taxonomy outlined by [Min *et al.*, 2022], most graph Transformers are designed from the perspectives of (1) incorporating the topology encoding into the node features, (2) incorporating the topology encoding into the attention matrix, and (3) utilizing graph neural networks [Wu *et al.*, 2021] as auxiliary modules.

Interestingly, it is well-known in both the graph learning [Chen *et al.*, 2022] and natural language processing communities [Zaheer *et al.*, 2020; Wang *et al.*, 2020] that, from

[1] https://numpy.org/doc/stable/index.html

the message-passing perspective, the key idea of the Transformer architecture is to reconstruct a weighted complete graph, whose adjacency matrix is $\mathbf{P} = \sigma\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d}}\right)$.

2.3 Invariant Rationale Discovery on Graphs

The graph rationale is a subgraph that encodes most downstream task-relevant semantics. A typical example is the functional groups in polymer graphs [Liu *et al.*, 2022; Wu *et al.*, 2022a], which fundamentally determines the chemical property of polymers. Mathematically, a given graph is decomposed into a rationale graph and an environment graph: $\mathcal{G} = \mathcal{G}_{\texttt{ra}} \cup \mathcal{G}_{\texttt{env}}$. Commonly, the graph embeddings on $\mathcal{G}_{\texttt{ra}}$ and $\mathcal{G}_{\texttt{env}}$ are computed as $\mathbf{h}_{\texttt{ra}}$ and $\mathbf{h}_{\texttt{env}}$. To ensure the rationale graph is invariant w.r.t. the prediction results when confronting different environments, a utility loss is minimized given the rationale embedding $\mathbf{h}_{\texttt{ra}}$ intervened by the environment embedding $\tilde{\mathbf{h}}_{\texttt{env}}$, i.e., $\min \mathcal{L}_{\texttt{util}}(\mathbf{h}_{\texttt{ra}} \xleftarrow{\text{intervene}} \tilde{\mathbf{h}}_{\texttt{env}})$. Here, $\tilde{\mathbf{h}}_{\texttt{env}}$ could be either from the same graph (i.e., $\tilde{\mathbf{h}}_{\texttt{env}} = \mathbf{h}_{\texttt{env}}$), or could be environment embeddings from other graphs, such as those in the batch. A key difference among existing methods lies in the intervention operation, of which we mention two:

- GREA [Liu *et al.*, 2022] designs the intervention as adding operation, i.e., $\mathbf{h}_{\texttt{ra}} + \tilde{\mathbf{h}}_{\texttt{env}}$;
- DIR [Wu *et al.*, 2022a] designs the intervention as an element-wisely weighting of prediction: $\theta_{\texttt{pred}}(\mathbf{h}_{\texttt{ra}}) \odot \texttt{Sigmoid}(\theta_{\texttt{pred}}(\tilde{\mathbf{h}}_{\texttt{env}}))$, where $\odot$ is the Hadamard product and $\theta_{\texttt{pred}}$ is a predictor.

The above intervention is conducted at the graph level because $\mathbf{h}_{\texttt{ra}}$ and $\tilde{\mathbf{f}}_{\texttt{env}}$ are graph embeddings. In Figure 2a, an overview of the GREA [Liu *et al.*, 2022] is presented. As a comparison, we aim to design the intervention at finer grains (e.g., node level) to handle the interaction between rationale and environment graphs, which will be detailed as follows.

3 Proposed Model

In this section, we introduce our proposed graph rationalization method, FIG. At its core, FIG utilizes a module based on the Transformer architecture. Figure 2b provides an overview of FIG, highlighting its four main parametric modules: the encoder, augmenter, intervener, and predictor.

Encoder. The encoder, denoted as $\theta_{\texttt{enc}} : \mathcal{G} \to \mathbb{R}^{n\times d}$, accepts a graph data as input and produces a node embedding matrix as output. While there are various graph encoders available, such as graph neural networks (GNNs) [Wu *et al.*, 2021] and graph Transformers [Min *et al.*, 2022]. From the methodology perspective, the encoder module is not the main contribution of this paper, so in this section, we do not specify a specific graph encoder $\theta_{\texttt{enc}}$.

Predictor. The predictor, denoted as $\theta_{\texttt{pred}} : \mathbb{R}^d \to \mathbb{R}^c$ takes as input a graph embedding and outputs a task-related vector/scalar. For graph regression tasks, $c = 1$; for graph classification tasks, c is the number of classes. A typical choice of predictor is a multi-layer perceptron (MLP) with appropriate activation functions. Details of the encoder and predictor in our implementation are presented in Section 4.

In subsequent subsections, we will elaborate on the augmenter and intervener, two essential modules. Their detailed designs derive two variants of the proposed FIG model.

3.1 Node-Level Variant: FIG-N

Node-level augmenter. The augmenter is a critical module of the proposed FIG. For the node-level variant, termed FIG-N, the augmenter's primary function is decomposing the node set into two distinct subsets: rationale nodes and environment nodes. This decomposition is operated by parameterizing the node-level augmenter as a learnable node partitioner, denoted by $\theta_{\texttt{aug-N}}$,

$$\mathbf{m} = \texttt{Sigmoid}(\texttt{MLP}(\mathbf{H}, \theta_{\texttt{aug-N}})), \tag{4}$$

whose input is the node embedding matrix $\mathbf{H} \in \mathbb{R}^{n\times d}$, and its output is a partition vector $\mathbf{m} \in [0,1]^n$. MLP is a multi-layer perceptron. Each entry within $\mathbf{m}$, such as $\mathbf{m}[i]$, signifies the probability of the i-th node being categorized as a rationale node.

For the node partition vector $\mathbf{m}$, its top-K entries are indexed as $\texttt{idx}_{\texttt{ra}} = \texttt{argtopK}(\mathbf{m})$ which is used to index the rationale nodes from the node embedding matrix $\mathbf{H}$; naturally, the remaining nodes are categorized as the environment nodes whose indices are $\texttt{idx}_{\texttt{env}} = \{1, \ldots, n\} - \texttt{idx}_{\texttt{ra}}$. K is a hyper-parameter whose impact is studied in Section D.1. Also, in our implementation, we use a soft argtopK operation to remain differentiability whose details are in Section E.

Using the indices mentioned above, rationale and environment embeddings, denoted as $\mathbf{H}_{\texttt{ra}}$ and $\mathbf{H}_{\texttt{env}}$, respectively, can be extracted from the node embedding matrix $\mathbf{H}$:

$$\mathbf{H}_{\texttt{ra}} = \mathbf{H}[\texttt{idx}_{\texttt{ra}}, :] \in \mathbb{R}^{K\times d}, \tag{5a}$$

$$\mathbf{H}_{\texttt{env}} = \mathbf{H}[\texttt{idx}_{\texttt{env}}, :] \in \mathbb{R}^{(n-K)\times d}, \tag{5b}$$

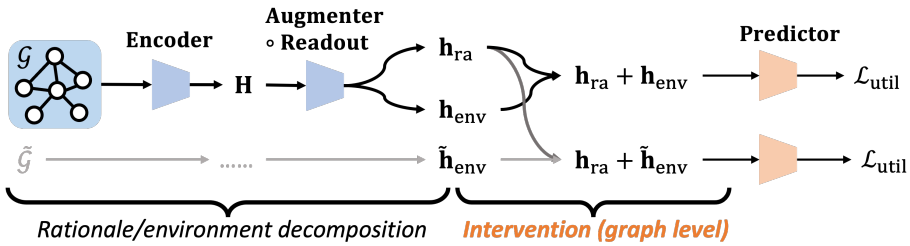
Node-level intervener. The design of the fine-grained intervener draws inspiration from the Transformer architecture [Vaswani *et al.*, 2017]. Explicitly, the node-level intervener ϕ is presented as,

$$\mathbf{H}_{\texttt{inter}}, \mathbf{P} = \texttt{Transformer}(\mathbf{H}_{\texttt{ra}} || \mathbf{H}_{\texttt{env}}), \tag{6a}$$

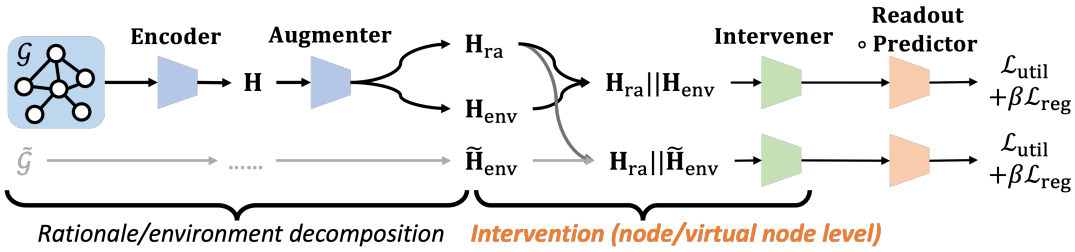
$$\textit{where} \quad \mathbf{P} = \texttt{Attn}(\mathbf{H}_{\texttt{ra}} || \mathbf{H}_{\texttt{env}}). \tag{6b}$$

In this representation, the operator $||$ concatenates along the first dimension of the matrices $\mathbf{H}_{\texttt{ra}}$ and $\mathbf{H}_{\texttt{env}}$. We dub the Eqs. (1)-(3) as Transformer and $\mathbf{P}$ is the intermediate attention matrix from the self-attention layer (Eq. (6b)). Here, the self-attention module models the interactions between the rational nodes $\mathbf{H}_{\texttt{ra}}$ and the environment nodes $\mathbf{H}_{\texttt{env}}$. ϕ includes all the parameters of the Attn (Eq. (6b)) and FFN (Eq. (3)) modules. In some contexts where the attention matrix $\mathbf{P}$ is not explicitly used as an output, input/output of the intervener ϕ can be presented as $\mathbf{H}_{\texttt{inter}} = \phi(\mathbf{H}_{\texttt{ra}} || \mathbf{H}_{\texttt{env}})$.

FIG-N optimization objective. The utility loss is computed as $\mathcal{L}_{\texttt{util}}(\mathbf{H}_{\texttt{ra}} || \mathbf{H}_{\texttt{env}}) = \mathcal{L}_{\texttt{task}}(\theta_{\texttt{pred}} \circ \texttt{Readout} \circ \phi(\mathbf{H}_{\texttt{ra}} || \mathbf{H}_{\texttt{env}}), \mathbf{y})$, where $\mathcal{L}_{\texttt{task}}$ is the task-specific objective. For instance, it could be the mean squared error for regression tasks or the cross-entropy for classification tasks. As introduced in Figure 1, the core of the invariant rationale discovery is to find the graph rationale so that the utility loss



(a) GREA [Liu *et al.*, 2022]



(b) FIG (Ours)

Figure 2: Pipeline comparison between existing work GREA and proposed FIG. $\circ$ denotes function composition. GREA designs the intervention at the graph level, and the proposed FIG designs the intervention at the node/virtual node level. The augmented environment $\tilde{\mathbf{H}}_{\texttt{env}}$ is from another graph $\tilde{\mathcal{G}}$ (through the Encoder and Augmenter) in the batch.

attains minimization given *changing environments*. Thus, the total utility objective is

$$\mathcal{L}_{\texttt{util}} = \mathcal{L}_{\texttt{util}}(\mathbf{H}_{\texttt{ra}}||\mathbf{H}_{\texttt{env}}) + \alpha\mathcal{L}_{\texttt{util}}(\mathbf{H}_{\texttt{ra}}||\tilde{\mathbf{H}}_{\texttt{env}}), \quad (7)$$

where $\tilde{\mathbf{H}}_{\texttt{env}}$ is the node embeddings from the changing environments. In practical implementations, $\tilde{\mathbf{H}}_{\texttt{env}}$ is the environment node embeddings from other graphs in the mini-batch. Additionally, to fully utilize the rich interactions from the fine-grained intervention module, we apply the following partition regularization term,

$$\mathcal{L}_{\texttt{reg}}(\mathbf{H}_{\texttt{ra}}||\mathbf{H}_{\texttt{env}}) = \mathbf{s}^\top\mathbf{P}(\mathbf{1}-\mathbf{s}) + (\mathbf{1}-\mathbf{s})^\top\mathbf{P}\mathbf{s}, \quad (8)$$

where $\mathbf{P} \in \mathbb{R}^{n\times n}$ is the self-attention matrix from Eq. (6b),

$$\mathbf{s}[i] = \begin{cases} 1 & \text{if } i < K. \\ 0 & \text{otherwise.} \end{cases} \quad (9)$$

0-based indexing is used so there are in total K non-zero entries (i.e., 1) in $\mathbf{s}$. The meaning of the binary $\mathbf{s}$ vector is to designate whether a particular row of the matrix $\mathbf{H}_{\texttt{ra}}||\mathbf{H}_{\texttt{env}}$ originates from the rationale nodes or the environment nodes. The underlying notion of the regularization term Eq. (8) is to impose penalties on interactions between the rationale nodes and the environment nodes. Namely, these two terms $\mathbf{s}^\top\mathbf{P}(\mathbf{1}-\mathbf{s})$ and $(\mathbf{1}-\mathbf{s})^\top\mathbf{P}\mathbf{s}$ denote the total weights on the links (i.e., cut) between the rationale and environment subgraphs. To handle the changing environments, we introduce an additional regularization term on the changing environments as $\mathcal{L}_{\texttt{reg}}(\mathbf{H}_{\texttt{ra}}||\tilde{\mathbf{H}}_{\texttt{env}})$ where $\tilde{\mathbf{H}}_{\texttt{env}}$ is the environment node embeddings from another graph within the same mini-batch. Then, the total regularization term is

$$\mathcal{L}_{\texttt{reg}} = \mathcal{L}_{\texttt{reg}}(\mathbf{H}_{\texttt{ra}}||\mathbf{H}_{\texttt{env}}) + \mathcal{L}_{\texttt{reg}}(\mathbf{H}_{\texttt{ra}}||\tilde{\mathbf{H}}_{\texttt{env}}), \quad (10)$$

and the total objective function is $\mathcal{L}_{\texttt{util}} + \beta\mathcal{L}_{\texttt{reg}}$. To fully harness the capabilities of the fine-grained parametric intervener, it is crucial to note—as highlighted in the introduction—that the behavior of the intervener ϕ operates in an adversarial fashion to the other modules. As a result, we formulate a min-max game that involves $\theta = \{\theta_{\texttt{enc}}, \theta_{\texttt{aug-N}}, \theta_{\texttt{pred}}\}$ and ϕ as,

$$\min_{\theta}\max_{\phi} \quad \mathcal{L}_{\texttt{util}} + \beta\mathcal{L}_{\texttt{reg}}. \quad (11)$$

Here, the intervener ϕ is trained to decrease the utility of the graph rationale by promoting interactions between the rationale nodes and the environment nodes. Conversely, the encoder, augmenter, and predictor (i.e., θ) are optimized in an opposing manner to the intervener's objectives.

Complexity of FIG-N. As the encoder $\theta_{\texttt{enc}}$ and the predictor $\theta_{\texttt{pred}}$ are off-the-shelf, the FIG-N introduces two new modules: the node-level augmenter, $\theta_{\texttt{aug-N}}$, and the Transformer-based intervener, ϕ. Notably, despite these additions, the increase in the number of parameters remains modest. The parameters for $\theta_{\texttt{aug-N}}$ originate from the `MLP` defined in Eq. (4). In a configuration where the `MLP` has 3 layers with a feature dimension of d, the parameter count is $O(2d^2)$. The intervener ϕ, driven by the `Transformer` layer in Eq. (6a), has its parameters confined to $O(3d^2 + 2d^2) = O(5d^2)$, owing to its query, key, value projection matrices and the feed-forward net (`FFN` from Eq. (3), typically a 3-layered `MLP`).

A step-by-step algorithm for FIG-N is in Algorithm 1. In test phase, the output of $\theta_{\texttt{pred}} \circ \texttt{Readout} \circ \phi(\mathbf{H}_{\texttt{ra}}||\mathbf{H}_{\texttt{env}})$ is evaluated.

3.2 Virtual Node-Level Variant: FIG-VN

In the previously introduced FIG-N, its augmenter decomposes the nodes into rationale nodes and environment nodes

Algorithm 1: FIG-N single training step for every training graph $\mathcal{G}$

Input : a labelled graph $(\mathcal{G}, y)$, a sampled graph $\tilde{\mathcal{G}}$ from the same batch as $\mathcal{G}$, $\theta = \{\theta_{\texttt{enc}}, \theta_{\texttt{aug-N}}, \theta_{\texttt{pred}}\}$, ϕ;

Output: updated θ and ϕ;

1 compute $\mathbf{H} = \theta_{\texttt{enc}}(G)$ and $\tilde{\mathbf{H}} = \theta_{\texttt{enc}}(\tilde{G})$;

2 compute $(\mathbf{H}_{\texttt{ra}}, \mathbf{H}_{\texttt{env}}) = \theta_{\texttt{aug-N}}(\mathbf{H})$, $(\tilde{\mathbf{H}}_{\texttt{ra}}, \tilde{\mathbf{H}}_{\texttt{env}}) = \theta_{\texttt{aug-N}}(\tilde{\mathbf{H}})$ via Eqs. (4), (5a), and (5b);

3 concatenate rationale-environment pairs $\mathbf{H}_{\texttt{ra}}||\mathbf{H}_{\texttt{env}}$ and $\mathbf{H}_{\texttt{ra}}||\tilde{\mathbf{H}}_{\texttt{env}}$;

4 compute $\mathcal{L}_{\texttt{util}}$ via Eq. (7);

5 compute $\mathcal{L}_{\texttt{reg}}$ via Eqs. (10) and (9);

6 update θ via gradient descent with $\frac{\partial(\mathcal{L}_{\texttt{util}}+\beta\mathcal{L}_{\texttt{reg}})}{\partial\theta}$;

7 update ϕ via gradient ascent with $\frac{\partial(\mathcal{L}_{\texttt{util}}+\beta\mathcal{L}_{\texttt{reg}})}{\partial\phi}$;

via a trainable node partitioner $\theta_{\texttt{aug-N}}$ so that the interaction is conducted at the node level (Eqs. (6a)) whose dense attention matrix's complexity is quadratic in terms of the node numbers, infeasible for large graphs. This section extends this idea to extract the graph rationale at the virtual node level, which has a lower computation complexity compared to FIG-N and provides an intermediate intervention granularity between the node-level model (FIG-N) and the graph-level model (GREA [Liu *et al.*, 2022]).

Virtual node-level augmenter. Our idea is partly inspired by the speedup technique from Linformer [Wang *et al.*, 2020], which reformulates both the attention matrix and node (token) embedding matrix to dimensions of $\mathbb{R}^{n\times r}$ and $\mathbb{R}^{r\times d}$, respectively. This reformulation ensures that their multiplication scales linearly with the number of nodes (tokens) n. Within this configuration, r, a pre-defined rank, is significantly smaller than n, and d represents the feature dimension. Drawing from the Linformer technique, we propose that the restructured token embedding matrix, with dimensions of $\mathbb{R}^{r\times d}$, can be interpreted as embeddings for r virtual nodes.

Building upon this insight, given node embeddings $\mathbf{H}$ from the encoder, the virtual node embeddings are:

$$\mathbf{H}_{\texttt{VN}} = \texttt{Softmax}(\mathbf{W}_{\texttt{N-VN}})\mathbf{H}. \tag{12}$$

Here, the row-wise applied `Softmax` function, along with $\texttt{Softmax}(\mathbf{W}_{\texttt{N-VN}}) \in \mathbb{R}^{r\times n}$, yields a trainable matrix assigning n nodes to r virtual nodes, where r acts as a tunable hyper-parameter. In experiments, we set $r = 8$. As all the virtual node embeddings are learned, a subset of the r virtual nodes can be designated as rationale virtual nodes, whose rationality is data-driven by the intervention procedure discussed in subsequent subsections. For brevity, the initial K virtual nodes are deemed as rationale virtual nodes, while the last $r - K$ nodes is considered the environment virtual node. Like the FIG-N, here K is a hyperparameter whose impact is studied in Section D.1. Thus, rationale and environment embeddings are presented as:

$$\mathbf{H}_{\texttt{ra}} = \mathbf{H}_{\texttt{VN}}[: K, :] \in \mathbb{R}^{K\times d}, \tag{13a}$$

$$\mathbf{H}_{\texttt{env}} = \mathbf{H}_{\texttt{VN}}[K :, :] \in \mathbb{R}^{(r-K)\times d}. \tag{13b}$$

The parameter of $\theta_{\texttt{aug-VN}}$ is $\mathbf{W}_{\texttt{N-VN}}$.

Virtual node-level intervener. This section discusses the design of a virtual node-level intervener, which parallels the framework presented in Section 3.1. The salient difference lies in that the intervention here functions on the virtual nodes rather than the given real nodes. Building upon our previous steps, we obtain the rationale virtual node embeddings, $\mathbf{H}_{\texttt{ra}} \in \mathbb{R}^{K\times d}$, and the environment node embedding, $\mathbf{H}_{\texttt{env}} \in \mathbb{R}^{(r-K)\times d}$. Thanks to the property of the Transformer that it can process sets with variable size, the design of the virtual node-level intervener ϕ is similar to the node-level intervener as $\mathbf{H}_{\texttt{inter}}, \mathbf{P} = \texttt{Transformer}(\mathbf{H}_{\texttt{ra}}||\mathbf{H}_{\texttt{env}})$ or short as $\mathbf{H}_{\texttt{inter}} = \phi(\mathbf{H}_{\texttt{ra}}||\mathbf{H}_{\texttt{env}})$ if the attention matrix $\mathbf{P}$ is not used. Notably, for FIG-VN, $\mathbf{P} \in \mathbb{R}^{r\times r}$ describes the interaction among the r virtual nodes.

FIG-VN optimization objective. The output of $\theta_{\texttt{pred}} \circ \texttt{Readout} \circ \phi(\cdot)$ is used for minimizing the utility loss $\mathcal{L}_{\texttt{util}} = \mathcal{L}_{\texttt{util}}(\mathbf{H}_{\texttt{ra}}||\mathbf{H}_{\texttt{env}}) + \alpha\mathcal{L}_{\texttt{util}}(\mathbf{H}_{\texttt{ra}}||\tilde{\mathbf{H}}_{\texttt{env}})$, where $\mathcal{L}_{\texttt{util}}(\mathbf{H}_{\texttt{ra}}||\mathbf{H}_{\texttt{env}}) = \mathcal{L}_{\texttt{task}}(\theta_{\texttt{pred}} \circ \texttt{Readout} \circ \phi(\mathbf{H}_{\texttt{ra}}||\mathbf{H}_{\texttt{env}}), \mathbf{y})$ and $\mathcal{L}_{\texttt{util}}(\mathbf{H}_{\texttt{ra}}||\tilde{\mathbf{H}}_{\texttt{env}})$ is defined similarly. For modeling the changing environment, $\tilde{\mathbf{H}}_{\texttt{env}}$ is the virtual node embeddings from other graphs in the mini-batch. Additionally, the previously proposed regularization term Eq, (8) can be extended to the virtual node-level variant: $\mathcal{L}_{\texttt{reg}}(\mathbf{H}_{\texttt{ra}}||\mathbf{H}_{\texttt{env}}) = \mathbf{s}^\top\mathbf{P}(\mathbf{1}-\mathbf{s}) + (\mathbf{1}-\mathbf{s})^\top\mathbf{P}\mathbf{s}$. The total regularization term, considering the changing environment $\tilde{\mathbf{H}}_{\texttt{env}}$, is $\mathcal{L}_{\texttt{reg}} = \mathcal{L}_{\texttt{reg}}(\mathbf{H}_{\texttt{ra}}||\mathbf{H}_{\texttt{env}}) + \mathcal{L}_{\texttt{reg}}(\mathbf{H}_{\texttt{ra}}||\tilde{\mathbf{H}}_{\texttt{env}})$. As the $\mathbf{P}$ depicts interactions among virtual nodes, we construct the rationale/environment indicator vector $\mathbf{s}$ analogously to Eq. (9). Put everything together, and the optimization objective of FIG-VN is $\min_\theta \max_\phi \ \mathcal{L}_{\texttt{util}} + \beta\mathcal{L}_{\texttt{reg}}$, where $\theta = \{\theta_{\texttt{enc}}, \theta_{\texttt{aug-VN}}, \theta_{\texttt{pred}}\}$.

Complexity of FIG-VN. As we mentioned the encoder $\theta_{\texttt{enc}}$ and the predictor $\theta_{\texttt{pred}}$ are off-the-shelf. Thus, the extra modules introduced by the FIG-VN are the virtual node-level augmenter $\theta_{\texttt{aug-VN}}$ and the Transformer-based intervener ϕ. The parameters for $\theta_{\texttt{aug-VN}}$ originate from the matrix $\mathbf{W}_{\texttt{N-VN}}$, as defined in Eq. (12). The number of these parameters is in order $O(nr)$, where n denotes the number of nodes. For practical implementation purposes, n is pre-set; it is set to $10\times$ average size of graphs from the dataset, and we truncate the input graphs if its size is larger than $10\times$ average size. The intervener ϕ parameters originate from the `Transformer` layer, outlined in Eq. (6a). The number of parameters here is $O(5d^2)$, owing to its query, key, value projection matrices, and the feed-forward net (Eq. (3), typically a 3-layered `MLP`).

A step-by-step algorithm for FIG-VN is in Algorithm 2, Appendix. In test phase, the output of $\theta_{\texttt{pred}} \circ \texttt{Readout} \circ \phi(\mathbf{H}_{\texttt{ra}}||\mathbf{H}_{\texttt{env}})$ is evaluated.

4 Experiments

In this section, we begin by detailing the datasets and baseline methods. Subsequent subsections evaluate the effectiveness of our approach, supplemented by an efficiency study,

Table 1: Effectiveness comparison (mean±std) with baseline methods. (↓) denotes the lower the better and (↑) denotes the higher the better. Statistics in grey are reported in the original papers. The best is bold, and the second best is underlined. N/A means the method cannot work on regression tasks.

	Graph Regression			Graph Classification			
Dataset	ZINC	AQSOL	mollipo	molhiv	moltox21	molbace	molbbbp
Metric	MAE(↓)	MAE(↓)	RMSE(↓)	AUC(↑)	AUC(↑)	AUC(↑)	AUC(↑)
GIN	0.350±0.008	1.237±0.011	0.783±0.017	77.1±1.5	75.6±0.9	80.7±1.2	69.5±1.0
GAT	0.723±0.010	1.638±0.048	0.923±0.011	75.0±0.5	72.2±0.6	75.3±0.8	67.1±0.6
GATv2	0.729±0.015	1.722±0.022	0.943±0.021	72.2±0.5	73.6±0.2	76.8±1.6	65.7±0.7
GatedGCN	0.579±0.023	1.533±0.035	0.819±0.033	74.8±1.6	75.0±0.8	81.2±1.2	68.3±0.9
GT	0.226±0.014	1.319±0.026	0.882±0.020	73.5±0.4	75.0±0.6	77.1±2.3	65.0±1.1
GraphiT	0.202±0.011	1.162±0.005	0.846±0.023	74.6±1.0	71.8±1.3	73.4±3.6	64.6±0.5
SAN	0.139±0.006	1.199±0.218	0.816±0.112	77.9±0.2	71.3±0.8	79.0±3.1	63.8±0.9
SAT	0.094±0.008	1.236±0.023	0.835±0.008	78.8±0.6	75.6±0.7	83.6±2.1	69.6±1.3
Graphormer	0.122±0.006	1.265±0.025	0.911±0.015	79.3±0.4	77.3±0.8	79.3±3.0	67.7±0.9
GraphTrans	0.192±0.011	1.233±0.052	0.915±0.032	78.1±0.5	76.4±0.8	78.0±1.8	70.5±0.9
GPS	**0.070±0.004**	1.032±0.007	0.780±0.021	78.8±1.0	75.7±0.4	79.6±1.4	69.6±1.1
DIR	N/A	N/A	N/A	77.1±0.6	73.1±0.2	74.8±0.3	70.5±1.4
GREA	0.227±0.020	1.177±0.019	0.769±0.025	79.3±0.9	78.2±0.9	82.4±2.4	69.9±1.8
FIG-N	0.095±0.008	**0.990±0.012**	0.708±0.013	80.1±0.7	**78.8±0.5**	**85.3±2.0**	**73.8±0.7**
FIG-VN	0.086±0.012	1.011±0.009	**0.706±0.009**	**80.2±1.0**	78.2±0.6	84.5±1.3	73.1±0.8

an ablation study, a sensitivity study, a visualization of the attention matrix, and a visualization of the rationales detected. Convergence analysis can be found in Appendix, Section D.2.

4.1 Setup

In this paper, we use 7 publicly-available real-world datasets: (1) graph classification datasets molhiv [Hu *et al.*, 2020], moltox21 [Hu *et al.*, 2020], molbace [Hu *et al.*, 2020], molbbbp [Hu *et al.*, 2020] and (2) graph regression datasets ZINC [Dwivedi *et al.*, 2023], AQSOL [Dwivedi *et al.*, 2023], and mollipo [Hu *et al.*, 2020]. We strictly follow the metrics and dataset split recommended by the given benchmarks. To be concrete, area under the ROC curve (AUC) is the metric for datasets molhiv, moltox21, molbace, molbbbp; root-mean-square deviation (RMSE) is the metric for dataset mollipo; mean absolute error (MAE) is the metric of datasets ZINC and AQSOL. The detailed statistics of the datasets are given in Table 5 (Appendix). We report the average result with the standard deviation in 10 runs.

Our baseline methods include (1) 4 graph neural network models: GIN [Xu *et al.*, 2019], GAT [Velickovic *et al.*, 2018], GATv2 [Brody *et al.*, 2022], and GatedGCN [Bresson and Laurent, 2017] (2) 7 graph Transformers: GT [Dwivedi and Bresson, 2020], GraphiT [Mialon *et al.*, 2021], SAN [Kreuzer *et al.*, 2021], SAT [Chen *et al.*, 2022], Graphormer [Ying *et al.*, 2021], GraphTrans [Wu *et al.*, 2022b], GPS [Rampásek *et al.*, 2022], and (3) 2 graph rationale discovery methods: DIR [Wu *et al.*, 2022a] and GREA [Liu *et al.*, 2022].

4.2 Effectiveness Study

The effectiveness comparison between the proposed FIG-N, FIG-VN, and baseline methods are provided in Table 1. To ensure a fair comparison, certain pre-trained models, such as the pre-trained Graphormer [Ying *et al.*, 2021], are omitted. As DIR [Wu *et al.*, 2022a] is designed to conduct interventions on the label prediction vectors, it cannot be directly applied to graph regression tasks.

Table 2: Number of parameter and FLOPs comparisons between the proposed augmenter, intervener, and common graph encoders.

Model	# Parameters	FLOPs
GIN	$1,708,807$	$53,008,220$
SAT	$2,790,739$	$101,520,116$
GraphTrans	$2,793,307$	$111,548,906$
GPS	$3,236,239$	$133,229,235$
$\{\theta_{\texttt{aug-N}}, \phi\}$	$453,001$	$31,303,800$
$\{\theta_{\texttt{aug-VN}}, \phi\}$	$363,320$	$14,558,400$

We have several observations. First, our proposed FIG-N and FIG-VN consistently outperform, or are at least on par with, all the baseline methods on the graph classification and regression datasets. Second, the virtual-node variant FIG-VN does not lead to significant performance degradation compared to the node-level variant FIG-N.

4.3 Efficiency Study

A detailed comparison regarding the number of parameters and the FLOPs (floating point operations) is presented in Table 2, where we list 4 typical 5-layered encoders (GIN, SAT, GraphTrans, and GPS), and our proposed node-/virtual node-level augmenter, intervener modules (i.e., $\{\theta_{\texttt{aug-N}}, \phi\}$ and $\{\theta_{\texttt{aug-VN}}, \phi\}$). The comparison shows that our proposed interveners are lightweight and only increase very minor computational costs.

We also compare the model efficiency (training iterations/second) of FIG-N and FIG-VN in Table 3 working with different encoders (GIN and GPS). The batch size is set as 32. We note that the inclusion of our proposed augmenter and intervener, represented as $\{\theta_{\texttt{aug-N}}, \phi\}$ or $\{\theta_{\texttt{aug-VN}}, \phi\}$, introduces a slight reduction in training speed. That is because

Table 3: Wall-clock time (iterations/second) comparison of different encoder-intervener combinations. The larger, the faster. (↓) denotes the speed degradation compared with the vanilla encoder.

Encoder	Intervener	mollipo	molbace	molbbbp
GIN	None	29.38	25.62	27.11
	FIG-N	23.05(↓6.33)	21.23(↓4.39)	21.61(↓5.50)
	FIG-VN	23.35(↓6.03)	21.46(↓4.16)	22.32(↓4.79)
GPS	None	24.29	20.51	22.30
	FIG-N	19.57(↓4.72)	17.83(↓2.68)	18.67(↓3.63)
	FIG-VN	19.93(↓4.63)	18.16(↓2.35)	18.88(↓3.42)

Table 4: Ablation study (mean±std) of the proposed model FIG. (↓) denotes the lower the better and (↑) denotes the higher the better.

Dataset Metric	mollipo RMSE(↓)	molbace AUC(↑)	molbbbp AUC(↑)
$\theta_{\texttt{enc}} \circ \theta_{\texttt{pred}}$	0.780±0.021	79.6±1.4	70.5±0.9
FIG-N w/o reg	0.736±0.022	84.3±0.7	72.3±1.0
FIG-VN w/o reg	0.758±0.018	83.2±1.5	71.9±0.7
FIG-N w/ reg	0.708±0.013	85.3±2.0	73.8±0.7
FIG-VN w/ reg	0.706±0.009	84.5±1.3	73.1±0.8

the proposed parametric augmenter and intervener increase the steps of the data pipeline, as presented in figure 2b, and enlarge the computational graph for auto-gradient tools, such as PyTorch. Fortunately, the parameter count of the parametric augmenter and intervener is low, ensuring that the overall training speed of the model is not dramatically affected.

4.4 Ablation Study

We conducted an ablation study on the proposed models, FIG-N and FIG-VN. We designed two ablated variants as baselines: (1) $\theta_{\texttt{enc}} \circ \theta_{\texttt{pred}}$ which is a pure composition of the encoder $\theta_{\texttt{enc}}$ and the predictor $\theta_{\texttt{pred}}$ without any rationale discovery module. Many of the existing graph classifiers are in this form, and here we select the GraphGPS [Rampásek *et al.*, 2022], which is also the backbone of our FIG model. (2) FIG-N w/o reg and FIG-VN w/o reg which remove the regularization term (Eq. (10)) from the objective function. Our results in Table 4 highlight that (1) equipped with the proposed Transformer-based intervener, the model's performance improves across all the datasets; e.g., the AUC is improved from 79.6% to 84.3% (FIG-N) on the molbace dataset. (2) With the proposed regularization term, the model's performance can be improved further; e.g., the AUC of the FIG-N is improved from 72.3% to 73.8% on the molbbbp dataset.

5 Related Work

This section introduces two topics related to this paper: (1) invariant learning on graphs and (2) Transformer on graphs.

Invariant Learning on Graphs. Invariant learning, which has extensive applications in many fields such as computer vision [Kaddour *et al.*, 2022; Arjovsky *et al.*, 2019; Ganin and Lempitsky, 2015; Chang *et al.*, 2020], is gaining more attention in the community of graph machine learning. OOD-GNN [Li *et al.*, 2023], for instance, applies random Fourier features to eliminate the statistical dependence between relevant and irrelevant graph representations. [Bevilacqua *et al.*, 2021] studies the graph invariant representation learning with a particular focus on the size discrepancies between the training/test graphs. DIR [Wu *et al.*, 2022a] decomposes the given graph into a rationale subgraph and an environment subgraph; after that, it uses a graph classifier to conduct prediction on the above two graphs respectively and its intervention is via the Hadamard product between the prediction vectors. Similarly, GREA [Liu *et al.*, 2022] conducts the rationale/environment decomposition at the node level and its intervention operation is to directly add the environment graph embedding into the rationale graph embedding. In a similar vein, GIL [Li *et al.*, 2022b] decomposes the given graph into the invariant and variant subgraphs; based on that, it infers the environment label, as input of the invariance regularization term [Koyama and Yamaguchi, 2020]. Furthermore, invariant learning can also benefit graph contrastive learning [Sui *et al.*, 2024] to enhance data augmentation. Using invariant learning to address the OOD generalization challenges on graph [Li *et al.*, 2022a; Gui *et al.*, 2022; Yue *et al.*, 2024; Sui *et al.*, 2023; Yao *et al.*, 2024] is promising, but our paper does not concentrate on this setting, and we leave it as the future work.

Transformer on Graphs. The Transformer architecture [Vaswani *et al.*, 2017] has achieved significant success in various domains, including natural language processing [Vaswani *et al.*, 2017], computer vision [Han *et al.*, 2023], and more [Xu *et al.*, 2022]. In recent years, there has been a surge of interest in enhancing graph machine learning methods by leveraging the capabilities of Transformers. For example, GraphTrans [Wu *et al.*, 2022b] concatenates the Transformer architecture after various message-passing neural networks; GPS [Rampásek *et al.*, 2022] proposes a powerful layer which operates both a graph convolution layer and a Transformer layer in parallel; GT [Dwivedi and Bresson, 2020] generalizes the attention module on graphs via concatenating the spectral vectors with the raw node features and computing the attention weights on all the existing edges; Graphormer [Ying *et al.*, 2021] encodes both the node centrality and node-pair shortest path into the Transformer architecture. From the architecture perspective, a comprehensive survey about graph Transformer can be found in [Min *et al.*, 2022].

6 Conclusion

This paper studies the invariant rationale discovery problem on graph data. Distinct from existing methods, our proposed solutions (FIG-N and FIG-VN) are designed at more fine-grained levels, specifically at the node and virtual node levels, so that they can better model the interactions between the rationale and environment subgraphs. More importantly, we formulate the intervener and the other model modules in a min-max game, which can significantly improve the quality of the extracted graph rationales. Comprehensive experiments on 7 real-world datasets illustrate the effectiveness of the proposed method compared to 13 baseline methods. Our evaluation of the proposed augmenter's and intervener's efficiency shows that they can largely retain the overall training efficiency.

References

[Arjovsky *et al.*, 2019] Martín Arjovsky, Léon Bottou, Ishaan Gulrajani, and David Lopez-Paz. Invariant risk minimization. *CoRR*, abs/1907.02893, 2019.

[Ba *et al.*, 2016] Lei Jimmy Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. Layer normalization. *CoRR*, abs/1607.06450, 2016.

[Bevilacqua *et al.*, 2021] Beatrice Bevilacqua, Yangze Zhou, and Bruno Ribeiro. Size-invariant graph representations for graph classification extrapolations. In Marina Meila and Tong Zhang, editors, *Proceedings of the 38th International Conference on Machine Learning, ICML 2021, 18-24 July 2021, Virtual Event*, volume 139 of *Proceedings of Machine Learning Research*, pages 837–851. PMLR, 2021.

[Bresson and Laurent, 2017] Xavier Bresson and Thomas Laurent. Residual gated graph convnets. *CoRR*, abs/1711.07553, 2017.

[Brody *et al.*, 2022] Shaked Brody, Uri Alon, and Eran Yahav. How attentive are graph attention networks? In *The Tenth International Conference on Learning Representations, ICLR 2022, Virtual Event, April 25-29, 2022*. OpenReview.net, 2022.

[Chang *et al.*, 2020] Shiyu Chang, Yang Zhang, Mo Yu, and Tommi S. Jaakkola. Invariant rationalization. In *Proceedings of the 37th International Conference on Machine Learning, ICML 2020, 13-18 July 2020, Virtual Event*, volume 119 of *Proceedings of Machine Learning Research*, pages 1448–1458. PMLR, 2020.

[Chen *et al.*, 2022] Dexiong Chen, Leslie O'Bray, and Karsten M. Borgwardt. Structure-aware transformer for graph representation learning. In Kamalika Chaudhuri, Stefanie Jegelka, Le Song, Csaba Szepesvári, Gang Niu, and Sivan Sabato, editors, *International Conference on Machine Learning, ICML 2022, 17-23 July 2022, Baltimore, Maryland, USA*, volume 162 of *Proceedings of Machine Learning Research*, pages 3469–3489. PMLR, 2022.

[Dwivedi and Bresson, 2020] Vijay Prakash Dwivedi and Xavier Bresson. A generalization of transformer networks to graphs. *CoRR*, abs/2012.09699, 2020.

[Dwivedi *et al.*, 2023] Vijay Prakash Dwivedi, Chaitanya K. Joshi, Anh Tuan Luu, Thomas Laurent, Yoshua Bengio, and Xavier Bresson. Benchmarking graph neural networks. *J. Mach. Learn. Res.*, 24:43:1–43:48, 2023.

[Ganin and Lempitsky, 2015] Yaroslav Ganin and Victor S. Lempitsky. Unsupervised domain adaptation by backpropagation. In Francis R. Bach and David M. Blei, editors, *Proceedings of the 32nd International Conference on Machine Learning, ICML 2015, Lille, France, 6-11 July 2015*, volume 37 of *JMLR Workshop and Conference Proceedings*, pages 1180–1189. JMLR.org, 2015.

[Gui *et al.*, 2022] Shurui Gui, Xiner Li, Limei Wang, and Shuiwang Ji. GOOD: A graph out-of-distribution benchmark. In *NeurIPS*, 2022.

[Han *et al.*, 2023] Kai Han, Yunhe Wang, Hanting Chen, Xinghao Chen, Jianyuan Guo, Zhenhua Liu, Yehui Tang, An Xiao, Chunjing Xu, Yixing Xu, Zhaohui Yang, Yiman Zhang, and Dacheng Tao. A survey on vision transformer. *IEEE Trans. Pattern Anal. Mach. Intell.*, 45(1):87–110, 2023.

[Hu *et al.*, 2020] Weihua Hu, Matthias Fey, Marinka Zitnik, Yuxiao Dong, Hongyu Ren, Bowen Liu, Michele Catasta, and Jure Leskovec. Open graph benchmark: Datasets for machine learning on graphs. *CoRR*, abs/2005.00687, 2020.

[Ioffe and Szegedy, 2015] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In Francis R. Bach and David M. Blei, editors, *Proceedings of the 32nd International Conference on Machine Learning, ICML 2015, Lille, France, 6-11 July 2015*, volume 37 of *JMLR Workshop and Conference Proceedings*, pages 448–456. JMLR.org, 2015.

[Kaddour *et al.*, 2022] Jean Kaddour, Aengus Lynch, Qi Liu, Matt J. Kusner, and Ricardo Silva. Causal machine learning: A survey and open problems. *CoRR*, abs/2206.15475, 2022.

[Koyama and Yamaguchi, 2020] Masanori Koyama and Shoichiro Yamaguchi. Out-of-distribution generalization with maximal invariant predictor. *CoRR*, abs/2008.01883, 2020.

[Kreuzer *et al.*, 2021] Devin Kreuzer, Dominique Beaini, William L. Hamilton, Vincent Létourneau, and Prudencio Tossou. Rethinking graph transformers with spectral attention. In Marc'Aurelio Ranzato, Alina Beygelzimer, Yann N. Dauphin, Percy Liang, and Jennifer Wortman Vaughan, editors, *Advances in Neural Information Processing Systems 34: Annual Conference on Neural Information Processing Systems 2021, NeurIPS 2021, December 6-14, 2021, virtual*, pages 21618–21629, 2021.

[Li *et al.*, 2022a] Haoyang Li, Xin Wang, Ziwei Zhang, and Wenwu Zhu. Out-of-distribution generalization on graphs: A survey. *CoRR*, abs/2202.07987, 2022.

[Li *et al.*, 2022b] Haoyang Li, Ziwei Zhang, Xin Wang, and Wenwu Zhu. Learning invariant graph representations for out-of-distribution generalization. In *NeurIPS*, 2022.

[Li *et al.*, 2023] Haoyang Li, Xin Wang, Ziwei Zhang, and Wenwu Zhu. OOD-GNN: out-of-distribution generalized graph neural network. *IEEE Trans. Knowl. Data Eng.*, 35(7):7328–7340, 2023.

[Liu *et al.*, 2022] Gang Liu, Tong Zhao, Jiaxin Xu, Tengfei Luo, and Meng Jiang. Graph rationalization with environment-based augmentations. In Aidong Zhang and Huzefa Rangwala, editors, *KDD '22: The 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, Washington, DC, USA, August 14 - 18, 2022*, pages 1069–1078. ACM, 2022.

[Mialon *et al.*, 2021] Grégoire Mialon, Dexiong Chen, Margot Selosse, and Julien Mairal. Graphit: Encoding graph structure in transformers. *CoRR*, abs/2106.05667, 2021.

[Min *et al.*, 2022] Erxue Min, Runfa Chen, Yatao Bian, Tingyang Xu, Kangfei Zhao, Wenbing Huang, Peilin Zhao, Junzhou Huang, Sophia Ananiadou, and Yu Rong. Transformer for graphs: An overview from architecture perspective. *CoRR*, abs/2202.08455, 2022.

[Rampásek *et al.*, 2022] Ladislav Rampásek, Michael Galkin, Vijay Prakash Dwivedi, Anh Tuan Luu, Guy Wolf, and Dominique Beaini. Recipe for a general, powerful, scalable graph transformer. In *NeurIPS*, 2022.

[Srivastava *et al.*, 2014] Nitish Srivastava, Geoffrey E. Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: a simple way to prevent neural networks from overfitting. *J. Mach. Learn. Res.*, 15(1):1929–1958, 2014.

[Sui *et al.*, 2023] Yongduo Sui, Qitian Wu, Jiancan Wu, Qing Cui, Longfei Li, Jun Zhou, Xiang Wang, and Xiangnan He. Unleashing the power of graph data augmentation on covariate distribution shift. In Alice Oh, Tristan Naumann, Amir Globerson, Kate Saenko, Moritz Hardt, and Sergey Levine, editors, *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*, 2023.

[Sui *et al.*, 2024] Yongduo Sui, Caizhi Tang, Zhixuan Chu, Junfeng Fang, Yuan Gao, Qing Cui, Longfei Li, Jun Zhou, and Xiang Wang. Invariant graph learning for causal effect estimation. In Tat-Seng Chua, Chong-Wah Ngo, Ravi Kumar, Hady W. Lauw, and Roy Ka-Wei Lee, editors, *Proceedings of the ACM on Web Conference 2024, WWW 2024, Singapore, May 13-17, 2024*, pages 2552–2562. ACM, 2024.

[Vaswani *et al.*, 2017] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In Isabelle Guyon, Ulrike von Luxburg, Samy Bengio, Hanna M. Wallach, Rob Fergus, S. V. N. Vishwanathan, and Roman Garnett, editors, *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, December 4-9, 2017, Long Beach, CA, USA*, pages 5998–6008, 2017.

[Velickovic *et al.*, 2018] Petar Velickovic, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. Graph attention networks. In *6th International Conference on Learning Representations, ICLR 2018, Vancouver, BC, Canada, April 30 - May 3, 2018, Conference Track Proceedings*. OpenReview.net, 2018.

[Wang *et al.*, 2020] Sinong Wang, Belinda Z. Li, Madian Khabsa, Han Fang, and Hao Ma. Linformer: Self-attention with linear complexity. *CoRR*, abs/2006.04768, 2020.

[Wu *et al.*, 2021] Zonghan Wu, Shirui Pan, Fengwen Chen, Guodong Long, Chengqi Zhang, and Philip S. Yu. A comprehensive survey on graph neural networks. *IEEE Trans. Neural Networks Learn. Syst.*, 32(1):4–24, 2021.

[Wu *et al.*, 2022a] Yingxin Wu, Xiang Wang, An Zhang, Xiangnan He, and Tat-Seng Chua. Discovering invariant rationales for graph neural networks. In *The Tenth International Conference on Learning Representations, ICLR 2022, Virtual Event, April 25-29, 2022*. OpenReview.net, 2022.

[Wu *et al.*, 2022b] Zhanghao Wu, Paras Jain, Matthew A. Wright, Azalia Mirhoseini, Joseph E. Gonzalez, and Ion Stoica. Representing long-range context for graph neural networks with global attention. *CoRR*, abs/2201.08821, 2022.

[Xu *et al.*, 2019] Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. How powerful are graph neural networks? In *7th International Conference on Learning Representations, ICLR 2019, New Orleans, LA, USA, May 6-9, 2019*. OpenReview.net, 2019.

[Xu *et al.*, 2022] Peng Xu, Xiatian Zhu, and David A. Clifton. Multimodal learning with transformers: A survey. *CoRR*, abs/2206.06488, 2022.

[Yao *et al.*, 2024] Tianjun Yao, Yongqiang Chen, Zhenhao Chen, Kai Hu, Zhiqiang Shen, and Kun Zhang. Empowering graph invariance learning with deep spurious infomax. In *Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21-27, 2024*. OpenReview.net, 2024.

[Ying *et al.*, 2021] Chengxuan Ying, Tianle Cai, Shengjie Luo, Shuxin Zheng, Guolin Ke, Di He, Yanming Shen, and Tie-Yan Liu. Do transformers really perform badly for graph representation? In Marc'Aurelio Ranzato, Alina Beygelzimer, Yann N. Dauphin, Percy Liang, and Jennifer Wortman Vaughan, editors, *Advances in Neural Information Processing Systems 34: Annual Conference on Neural Information Processing Systems 2021, NeurIPS 2021, December 6-14, 2021, virtual*, pages 28877–28888, 2021.

[Yue *et al.*, 2024] Linan Yue, Qi Liu, Ye Liu, Weibo Gao, Fangzhou Yao, and Wenfeng Li. Cooperative classification and rationalization for graph generalization. In Tat-Seng Chua, Chong-Wah Ngo, Ravi Kumar, Hady W. Lauw, and Roy Ka-Wei Lee, editors, *Proceedings of the ACM on Web Conference 2024, WWW 2024, Singapore, May 13-17, 2024*, pages 344–352. ACM, 2024.

[Zaheer *et al.*, 2020] Manzil Zaheer, Guru Guruganesh, Kumar Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontañón, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, and Amr Ahmed. Big bird: Transformers for longer sequences. In Hugo Larochelle, Marc'Aurelio Ranzato, Raia Hadsell, Maria-Florina Balcan, and Hsuan-Tien Lin, editors, *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual*, 2020.

[Zhang *et al.*, 2022] Zhenning Zhang, Boxin Du, and Hanghang Tong. Suger: A subgraph-based graph convolutional network method for bundle recommendation. In Mohammad Al Hasan and Li Xiong, editors, *Proceedings of the 31st ACM International Conference on Information & Knowledge Management, Atlanta, GA, USA, October 17-21, 2022*, pages 4712–4716. ACM, 2022.

Appendix of 'Fine-grained Graph Rationalization'

A Dataset Statistics

The statistics of the datasets are presented in Table 5.

Table 5: Dataset statistics.

Name	# Graphs	# Nodes	# Edges	# Features	# Classes	Split	Metric
ZINC	12000	23.2	49.8	21 (node), 4 (edge)	N/A	10000/1000/1000	MAE
AQSOL	9833	17.6	35.8	65 (node), 5 (edge)	N/A	7836/998/999	MAE
mollipo	4200	27.0	59.0	9 (node), 3 (edge)	N/A	3360/420/420	RMSE
molhiv	41127	25.5	54.9	9 (node), 3 (edge)	2	32901/4113/4113	AUC
moltox21	7831	18.6	38.6	9 (node), 3 (edge)	2	6264/783/784	AUC
molbace	1513	34.4	73.7	9 (node), 3 (edge)	2	1210/151/152	AUC
molbbbp	2039	24.1	51.9	9 (node), 3 (edge)	2	1631/204/204	AUC
molmuv	93087	24.2	52.6	9 (node), 3 (edge)	2	74469/9309/9309	AUC

B Hardware and Implementations.

We implement FIG-N, FIG-VN, and all the baseline methods in PyTorch[2] and PyTorch-geometric[3]. All the efficiency study results are from one NVIDIA Tesla V100 SXM2-32GB GPU on a server with 96 Intel(R) Xeon(R) Gold 6240R CPU @ 2.40GHz processors and 1.5T RAM. We directly use those statistics when baseline methods have pre-existing results for specific datasets. In cases where such results are absent, we implement the models based on either the available code or details described in the associated publications.

We first introduce some shared implementations among FIG-N/VN and baseline methods. The batch size is set as 32, and the weight decay is set as 0. The hidden dimension is set as 300. The dropout rate for both the encoder and the intervener is searched between $\{0, 0.5\}$. The pooling is searched between $\{\texttt{mean}, \texttt{sum}\}$. The normalization is searched between the batch normalization [Ioffe and Szegedy, 2015] and layer normalization [Ba *et al.*, 2016]. The learning rate is initialized as 0.0001 and will decay by a factor $\frac{1}{4}$ if the validation performance is not improved in 10 epochs. Next, we detail the implementation of FIG-N/VN and baseline methods.

B.1 Implementation of FIG-N/VN

The encoder is set as GPS [Rampásek *et al.*, 2022] on ZINC, AQSOL, mollipo, molhiv, molbace, and set as GraphTrans [Wu *et al.*, 2022b] on moltox21 and molbbbp. We follow the typical design for the predictor module as a 3-layered MLP with ReLU activation in the intermediate layers. In our implementation, we set $\beta = \frac{2\times\hat{\beta}}{n\times(n-1)}$ (FIG-N) or $\beta = \frac{2\times\hat{\beta}}{r\times(r-1)}$ (FIG-VN). The α and $\hat{\beta}$ are searched between $[0.2, 2]$, step size 0.2. In our implementation, the K is set as $K = \texttt{round}(\hat{K} \times n)$ (for FIG-N) or $K = \texttt{round}(\hat{K} \times r)$ (for FIG-VN). r is searched between $\{8, 16, 32\}$ for FIG-VN. We have a detailed sensitivity study to explore the best selection of $\hat{K}$ in Section D.1, which shows the best $\tilde{K}$ is around 0.75.

B.2 Implementation of Baseline Methods

We search the number of layers of GIN [Xu *et al.*, 2019], GAT [Velickovic *et al.*, 2018], GATv2 [Brody *et al.*, 2022], GatedGCN [Bresson and Laurent, 2017], DIR [Wu *et al.*, 2022a], and GREA [Liu *et al.*, 2022] between $\{2, 3, 5, 10\}$ and report the best performance, considering configurations both with and without a virtual node connecting to all the given nodes.

Regarding the Transformer-based baselines (GT [Dwivedi and Bresson, 2020], GraphiT [Mialon *et al.*, 2021], SAN [Kreuzer *et al.*, 2021], SAT [Chen *et al.*, 2022], Graphormer [Ying *et al.*, 2021], GraphTrans [Wu *et al.*, 2022b], GPS [Rampásek *et al.*, 2022]), for the (absolute or relative) positional encoding, we adhere to the suggestions made in their original papers. We also searched the number of layers between $\{2, 3, 5, 10\}$.

Our GIN, GAT, GATv2, and GatedGCN implementations are from the PyTorch-geometric package. Our implementations of

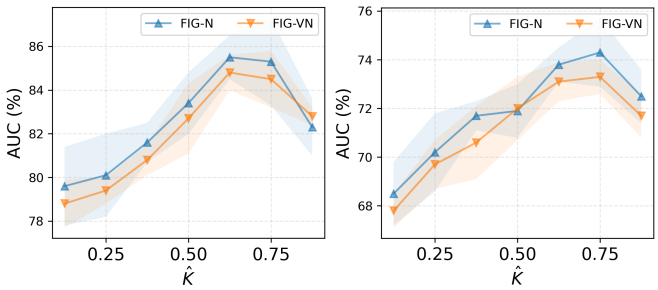
[2]https://pytorch.org/

[3]https://pytorch-geometric.readthedocs.io/en/latest/

Algorithm 2: FIG-VN single training step for every training graph $\mathcal{G}$

Input : a labelled graph $(\mathcal{G}, y)$, a sampled graph $\tilde{\mathcal{G}}$ from the same batch as $\mathcal{G}$, $\theta = \{\theta_{\texttt{enc}}, \theta_{\texttt{aug-VN}}, \theta_{\texttt{pred}}\}$, ϕ;
Output: updated θ and ϕ;

1. compute the node embedding matrices $\mathbf{H} = \theta_{\texttt{enc}}(G)$ and $\tilde{\mathbf{H}} = \theta_{\texttt{enc}}(\tilde{G})$;
2. compute rationale and environment embeddings as $(\mathbf{H}_{\texttt{ra}}, \mathbf{H}_{\texttt{env}}) = \theta_{\texttt{aug-VN}}(\mathbf{H})$, $(\tilde{\mathbf{H}}_{\texttt{ra}}, \tilde{\mathbf{H}}_{\texttt{env}}) = \theta_{\texttt{aug-VN}}(\tilde{\mathbf{H}})$ via Eqs. (12), (13a), and (13b);
3. concatenate rationale-environment pairs $\mathbf{H}_{\texttt{ra}}||\mathbf{H}_{\texttt{env}}$ and $\mathbf{H}_{\texttt{ra}}||\tilde{\mathbf{H}}_{\texttt{env}}$;
4. compute $\mathcal{L}_{\texttt{util}}$ via Eq. (7);
5. compute $\mathcal{L}_{\texttt{reg}}$ via Eqs. (10) and (9);
6. update θ via gradient descent with $\frac{\partial(\mathcal{L}_{\texttt{util}}+\beta\mathcal{L}_{\texttt{reg}})}{\partial\theta}$;
7. update ϕ via gradient ascent with $\frac{\partial(\mathcal{L}_{\texttt{util}}+\beta\mathcal{L}_{\texttt{reg}})}{\partial\phi}$;



(a) molbace (b) molbbbp

Figure 3: Performance of FIG-N/VN with different $\hat{K}$.

GT [4], GraphiT [5], SAN [6], SAT [7], Graphormer [8], GraphTrans [9], GPS [10], DIR [11], GREA [12] are adapted from publicly available code.

C Algorithms

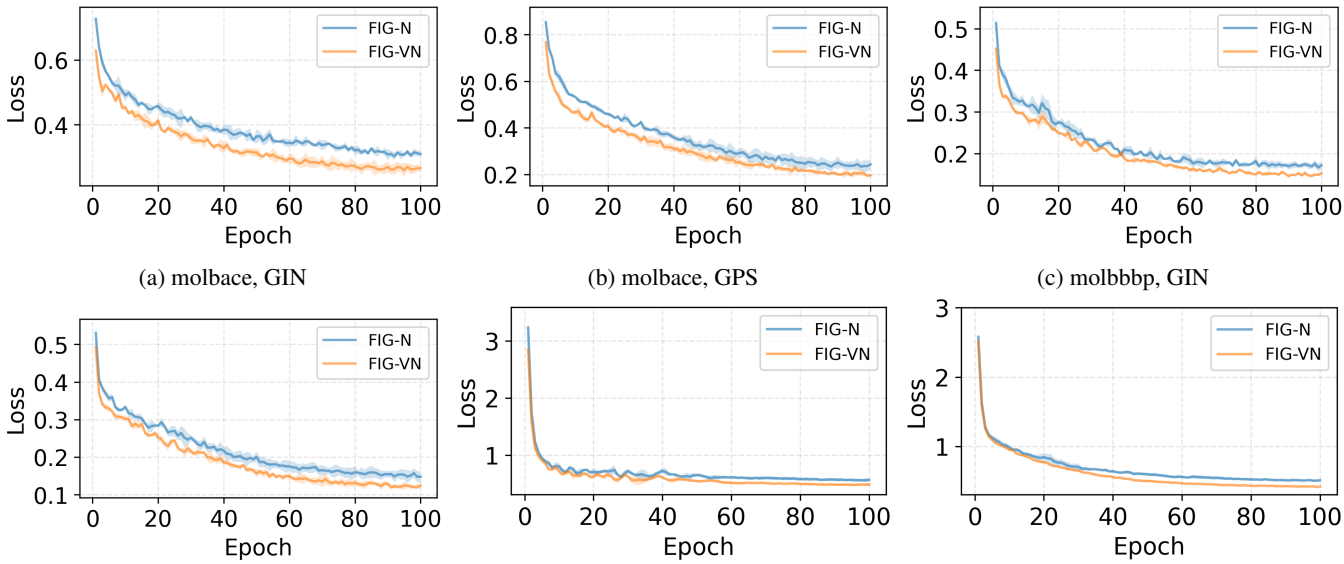
A step-by-step algorithm for FIG-VN is provided in Algorithm 2.

D Additional Experiments

D.1 Sensitivity Study

In this section, we carefully study the impact of hyperparameter K (from Eq. (5a), (5b), (13a), and (13b)), which determines the ratio of the rationale and environment subgraphs. In our implementation, we set $K = \texttt{round}(\hat{K} \times n)$ (for FIG-N) or $K = \texttt{round}(\hat{K} \times r)$ (for FIG-VN). We evaluate the model performance across varying $\hat{K}$ on the molbace and molbbbp datasets in Figure 3. We note that the model performance degrades if most nodes/virtual nodes are marked as the environment component. Similar performance degradation is observed if too many nodes/virtual nodes are marked as the rationale nodes (e.g., $\hat{K} = 0.875$). That is because for a large $\hat{K}$ (e.g., $\hat{K} = 1$), the model degenerates to a vanilla graph encoder, with less intervention involved. The best performance is observed when $\hat{K}$ is set as 0.75 or 0.675.

[4] https://github.com/graphdeeplearning/graphtransformer
[5] https://github.com/inria-thoth/GraphiT
[6] https://github.com/DevinKreuzer/SAN
[7] https://github.com/BorgwardtLab/SAT
[8] https://github.com/microsoft/Graphormer
[9] https://github.com/ucbrise/graphtrans
[10] https://github.com/rampasek/GraphGPS
[11] https://github.com/Wuyxin/DIR-GNN
[12] https://github.com/liugangcode/GREA



(a) molbace, GIN (b) molbace, GPS (c) molbbbp, GIN

(d) molbbbp, GPS (e) mollipo, GIN (f) mollipo, GPS

Figure 4: Training loss of FIG-N/VN with different datasets and encoders.

D.2 Training Convergence

We are concerned about the impact of the min-max objective on the training stability of FIG-N and FIG-VN. We monitor the training losses of both FIG-N and FIG-VN across three datasets (molbace, molbbbp, mollipo) using two encoders (GIN and GPS). The results, presented in Figure 4, demonstrate that the training remains stable even when θ (representing the encoder, augmenter, and predictor) and ϕ (representing the intervener) engage in a min-max game.

D.3 Attention Visualization

In this section, we aim to evaluate the significance of the regularization term by visualizing the adjacency matrix $\mathbf{P}$, effectively the attention matrix, of the intervener ϕ in Figure 5. For clarity in visualization, we choose FIG-VN. Unlike FIG-N, which works on a variable number of nodes (from different graphs), FIG-VN maps nodes to a predetermined number of virtual nodes, simplifying the presentation. Specifically, we set the number of virtual nodes r to 16 with K at 10, designating 10 virtual nodes to rationales and the remainder as environments. All the visualization results are obtained from the molbace dataset. It is worth noting that the attention matrix $\mathbf{P}$ is normalized row-wise by `Softmax`. From our observations, we highlight two primary insights:

- Interestingly, even in the absence of the regularization term, in Figure 5(a), interactions between rationales and environments appear significantly weaker than those within the rationales themselves. One potential explanation is the changing nature of the environment. In optimizing the utility loss $\mathcal{L}_{\texttt{util}} = \mathcal{L}_{\texttt{task}}(\mathbf{H}_{\texttt{ra}}||\mathbf{H}_{\texttt{env}}) + \alpha\mathcal{L}_{\texttt{task}}(\mathbf{H}_{\texttt{ra}}||\tilde{\mathbf{H}}_{\texttt{env}})$, the ever-changing environment ($\tilde{\mathbf{H}}_{\texttt{env}}$) might lead the model to minimize interactions between rationales and environments so that the utility of the rationale can be ensured.

- The first observation supports our decision to introduce the regularization term, which aims to penalize rationale-environment interactions. When the proposed regularization term (Eq. (8)) is implemented, in Figure 5(b), there is a noticeable decline in rationale-environment interactions (the off-diagonal blocks in Figure 5). As demonstrated in our earlier ablation study, this leads to enhanced model performance.

D.4 Rationale Visualization

Figure 6 presents examples showing the rationale detected by our FIG-N model on the AQSOL dataset [Dwivedi *et al.*, 2023].

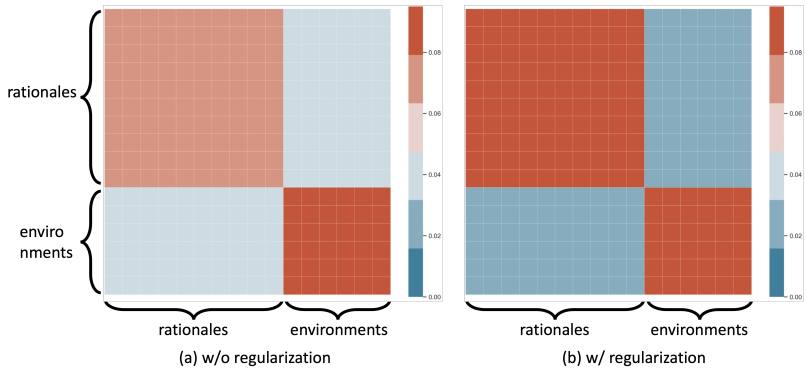


Figure 5: Heatmap of the adjacency matrix of the intervener ϕ. (a) without the regularization term Eq. (8) and (b) with the regularization term Eq. (8).

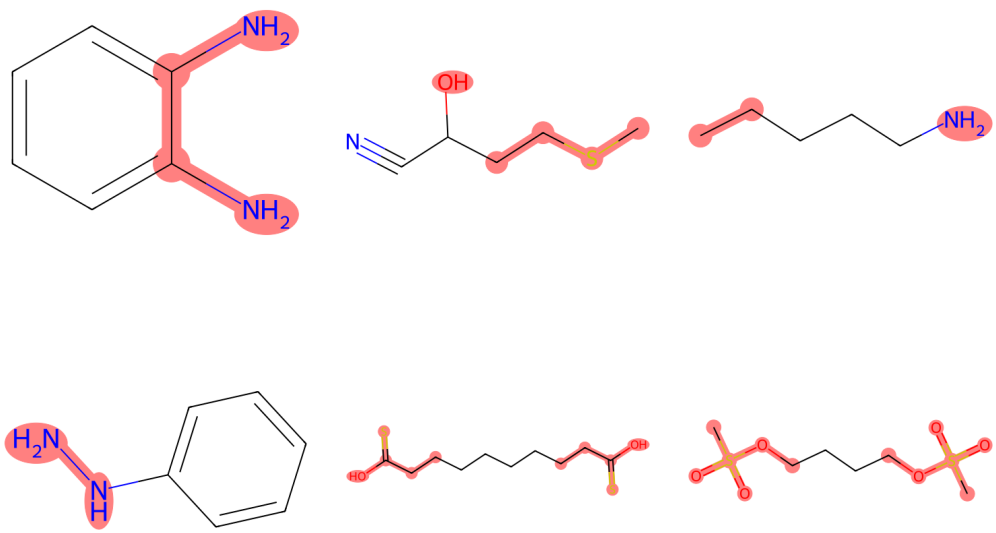


Figure 6: Rationales highlighted by FIG-N.

E Soft argtop-K Trick

In the main content, for the FIG-N model, the augmenter will generate the partition vector $\mathbf{m}$ which is then used to partition the node embedding matrix via selecting the top-K indices from $\mathbf{m}$:

$$\mathbf{m} = \texttt{Sigmoid}(\texttt{MLP}(\mathbf{H}, \theta_{\texttt{aug-N}})) \in \mathbb{R}^n \tag{14}$$

$$\texttt{idx}_{\texttt{ra}} = \texttt{argtopK}(\mathbf{m}) \in \mathbb{N}_+^K \tag{15}$$

$$\mathbf{H}_{\texttt{ra}} = \mathbf{H}[\texttt{idx}_{\texttt{ra}}, :] \in \mathbb{R}^{K \times d} \tag{16}$$

$$\mathbf{H}_{\texttt{env}} = \mathbf{H}[\texttt{idx}_{\texttt{env}}, :] \in \mathbb{R}^{(n-K) \times d} \tag{17}$$

The hard `argtopK` breaks the differentiability of the model so that $\theta_{\texttt{aug-N}}$ has no gradient. Here, we present a soft argtop-K trick which is inspired by the soft top-K trick[13]. Overall, the key ideas are as follows,

1. The index vector $\texttt{idx}_{\texttt{ra}} \in \mathbb{N}_+^K$ can be viewed as a list of one-hot index encoding $\mathbf{idx} \in \{0, 1\}^{K \times n}$ whose every row is a one-hot vector. If the i-th row's j-th element is 1, it means the j-th element in $\mathbf{m}$ is the i-th largest element in $\mathbf{m}$. Then,

[13] https://github.com/ZIB-IOL/merlin-arthur-classifiers/blob/main/soft-topk.py

we can use the matrix multiplication to index the matrix $\mathbf{H}$:

$$\mathbf{H}[\mathtt{idx}_{\mathtt{ra}},:] \Longleftrightarrow \mathbf{idx} \times \mathbf{H} \in \mathbb{R}^{K \times d} \tag{18}$$

2. We aim to find a soft and differentiable matrix to approximate $\mathbf{idx}$. The trick is to use the fact that every row of $\mathbf{idx}$ is one-hot, which can be approximated by the output of the softmax function. Hence, the implementation is to call the softmax K times repeatedly.
3. The parameterization trick[14] can be used: y_hard - y_soft.detach() + y_soft, whose main idea is to ensure (1) the forward process to use the hard indexing and (2) the backpropagation to update the soft indices.

We provide the exemplar code as follows.

```
import torch

def soft_arg_top_K(K,H,m):
    """
    Select the argtop K indices from the vector m, and index the node embedding matrix H.

    K: number of nodes selected
    H: node embedding matrix (n, d), n: number of nodes, d: hidden dimension
    m: masking vector (n,)
    """

    top_K_indices = []
    for i in range(K):
        top1_index_soft = torch.nn.Softmax(m)
        top1_index_hard = torch.zeros_like(top1_index_soft)
        top1_index_hard[torch.argmax(top1_index_soft)] = 1
        top1_index = top1_index_hard - top1_index_soft.detach() + top1_index_soft
        top_K_indices.append(top1_index)
        m = m - top1_index_hard * 1e6  # mask the selected entry
    top_K_indices = torch.stack(top_K_indices, dim=0)
    H_ra = torch.mm(top_K_indices, H)
    return H_ra
```

[14] https://pytorch.org/docs/stable/generated/torch.nn.functional.gumbel_softmax.html